

企业级应用软件设计与开发

Xcode

面向终端的 Agentic AI 原生开发平台

课程大作业报告

方向：Agentic AI 原生开发

学号：2025302985

姓名：龙新超

专业：计算机技术

指导教师：戚欣

提交日期：2026 年 6 月 22 日

目录

一、选题背景与设计思想

- 1.1 问题定义与原始系统分析
- 1.2 项目价值与技术路线

二、Specs 规格文档

- 2.1 Product Spec
- 2.2 Architecture Spec
- 2.3 API Spec

三、系统架构与设计

- 3.1 核心架构与 Agent 交互流程
- 3.2 Memory 与会话数据流

四、关键实现与代码展示

- 4.1 Agent 核心循环
- 4.2 Memory、/compact 与 /resume
- 4.3 Skill、MCP 与 QQChat

五、测试与评估

- 5.1 自动化测试与 Benchmark
- 5.2 Demo 截图占位

六、系统升级与扩展

七、课程总结

一、选题背景与设计思想

项目定位 Xcode 是一个运行在 Windows 终端中的本地 AI Coding Agent。它以同步 REPL 为主体，通过 OpenAI-compatible API 完成推理，并将文件、搜索、Shell、任务、Skill、MCP 与外部 QQ 入口统一到可审计的工具循环中。

1.1 问题定义与原始系统分析

传统代码助手通常停留在“问答 + 补全”层面，无法持续维护任务状态，也难以安全地调用本机工具。对于真实软件工程任务，系统必须同时处理长对话上下文、工具异常、权限审批、项目规则、会话恢复和多入口消息，这些能力若直接堆叠在单一主循环中，会造成状态污染、难以测试以及 Windows 终端兼容问题。

Xcode 的早期版本已经具备基础 LLM 调用与文件工具，但原始结构存在三类不足：一是上下文增长后只能依赖当前进程内历史，退出即丢失；二是长期规则、用户偏好和自动记忆没有明确分层；三是扩展能力容易直接侵入 AgentRuntime，增加主循环风险。

- 长对话：需要在不破坏 tool_calls/tool result 配对的前提下压缩和恢复。
- 长期记忆：需要区分项目规则、用户偏好与按主题沉淀的自动记忆。
- 生态扩展：Skill、MCP 和 QQChat 必须共享工具、安全与审计边界。
- 终端体验：方向键菜单、Rich 流式渲染和 Windows 路径必须在原生 PowerShell/cmd.exe 中验证。

1.2 项目价值与技术路线

本项目的价值不在于重新包装一个聊天界面，而在于构建一条可持续演进的 Agentic AI 工程链路：模型负责规划与决策，工具负责可观察的行动，权限系统约束副作用，session transcript 保存事实，memory 文件保存跨会话知识。

技术方向	方案	工程价值
核心运行时	同步 REPL + 持续 LLM/tool loop	降低异步传播复杂度，便于调试与审查

技术方向	方案	工程价值
上下文	token 估算 + 累积摘要 checkpoint	控制长对话成本并支持恢复
记忆	XCODE.md + MEMORY.md 文件 模型	可读、可编辑、可版本化
扩展	Skill prompt、MCP stdio、QQ external turn	按需扩展且保持边界清晰
安全	三级权限、trust gate、 ToolScope、只读标记	限制本机与远程入口的副作用

Xcode 核心架构

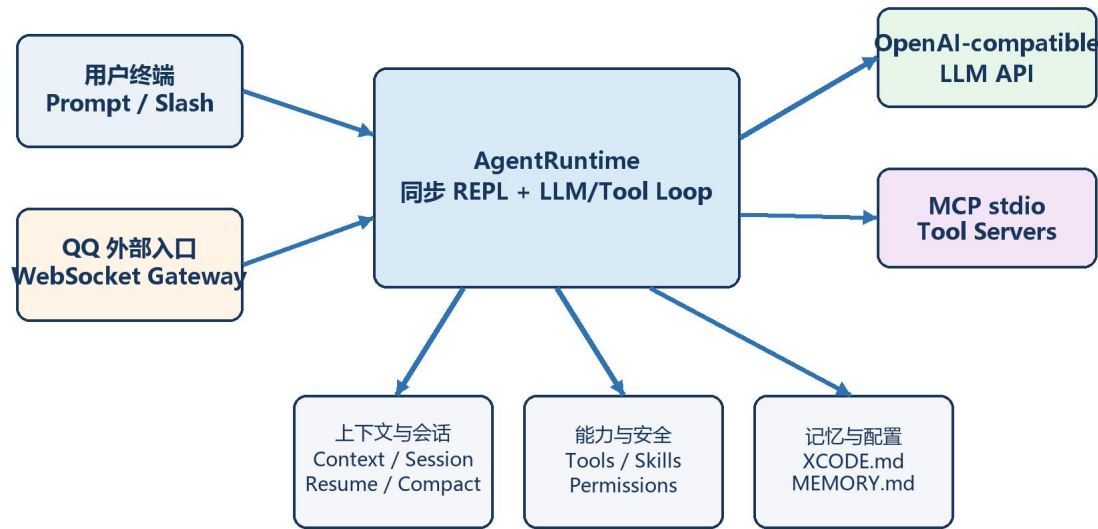


图 1 Xcode 项目技术路线与核心组件关系

二、Specs 规格文档

本章以 Software Design Description (SDD) 为核心，将产品目标、架构约束与关键接口规格统一描述。规格优先于实现，用于限定 Coding Agent 的实现范围和验收标准。

2.1 Product Spec

条目	规格
目标用户	在 Windows 终端中完成代码阅读、修改、测试与项目管理的开发者
核心场景	连续对话开发、会话恢复、上下文压缩、项目 Skill、MCP 工具、QQ 远程只读协作
关键输入	自然语言、slash command、项目文件、SKILL.md、MCP 配置、QQ 消息事件
关键输出	终端响应、工具执行结果、JSONL transcript、memory 文档、QQ 被动回复
成功指标	工具循环不因单个工具异常崩溃；恢复后可继续工作；扩展入口遵守权限边界

功能需求：

- FR-01：系统应将每轮 user/assistant/tool 消息追加写入项目 session transcript。
- FR-02：系统应在上下文达到预算 80% 时自动压缩，并允许用户手动执行 /compact。
- FR-03：/resume 应优先使用最新 checkpoint 与其后消息恢复；无 checkpoint 时使用预算内 recent tail。
- FR-04：系统应加载项目 XCODE.md、用户 XCODE.md 与自动记忆索引 MEMORY.md。
- FR-05：项目 Skill 应支持用户 slash 调用和模型按需调用，并记录来源与 source hash。
- FR-06：MCP server 未经项目绑定的 trust 校验不得启动；工具必须注册 is_read_only。
- FR-07：QQChat 默认只能使用只读工具，并为不同 conversation key 维护独立 history。

非功能需求：Python 版本不低于 3.10；主运行时不引入 asyncio；工具异常全部捕获；用户界面字符串使用中文；关键交互需在原生 PowerShell/cmd.exe 中验收。

2.2 Architecture Spec

层	主要模块	职责与约束
入口层	main.py、PromptSession、QQGatewayClient	接收本地或外部输入，不直接实现业务规则
编排层	AgentRuntime、ExternalTurnRunner	组织 user turn 与 LLM/tool loop，隔离会话状态
领域层	Context、Session、Memory、Skills、MCP、QQChat	各子系统只管理自身状态与策略
执行层	ToolRegistry、ToolCallExecutor、PermissionManager	统一 schema、审批、执行与异常收口
持久化层	JSONL、XCODE.md、MEMORY.md、本地 state/trust	append-only 事实与可读配置分离

- 约束 A：AgentRuntime 保持同步外观；MCP 仅允许在子系统内部使用 event loop/thread。
- 约束 B：外部入口不得复用本地 REPL 的 _history，也不得暴露写文件、Shell、子 Agent 与 Skill 工具。
- 约束 C：文件路径必须解析为真实 Windows 路径后再进行 memory 范围或 trust 判断。
- 约束 D：任何 server、工具或网络异常均转化为状态/工具错误，不能打断主 REPL。

2.3 API Spec

接口/命令	输入	输出/副作用	异常策略
ContextManager.co	messages、LLM client、previous	CompressionResult：摘要、压缩后	内容不足时原样返回；LLM 异

接口/命令	输入	输出/副作用	异常策略
mpress	summary	消息、checkpoint message	常由上层捕获
SessionResumeBuilder.build	transcript_path	ResumeResult: history、token 估算、checkpoint 标记	损坏 JSONL 行跳过；缺失文件返回空结果
MemoryManager.get_context_for_prompt	Config	项目/用户/索引三段 prompt context	缺失文件按空内容处理；总长度受预算限制
SkillInvocationService.invoke_for_model	skill_name、args	展开后的 prompt 与审计 metadata	非法、禁用或 fork skill 返回可读错误
MCPConnectionManager.call_tool_sync	server、tool、arguments	文本化 ToolOutput	超时、连接与 server 错误全部转换为工具错误
QQChatService.handle_gateway_event	QQ dispatch payload	normalize/policy/dedupe 后入队	回调只记录安全错误，不阻塞 gateway 线程
/compact	当前_history	替换 history 并追加 checkpoint	无可压缩内容显示 Nothing to compact
/resume	用户选择 session	恢复_history 并渲染最近对话	取消、失败、无 session 不修改当前历史

三、系统架构与设计

3.1 核心架构与 Agent 交互流程

Xcode 核心架构

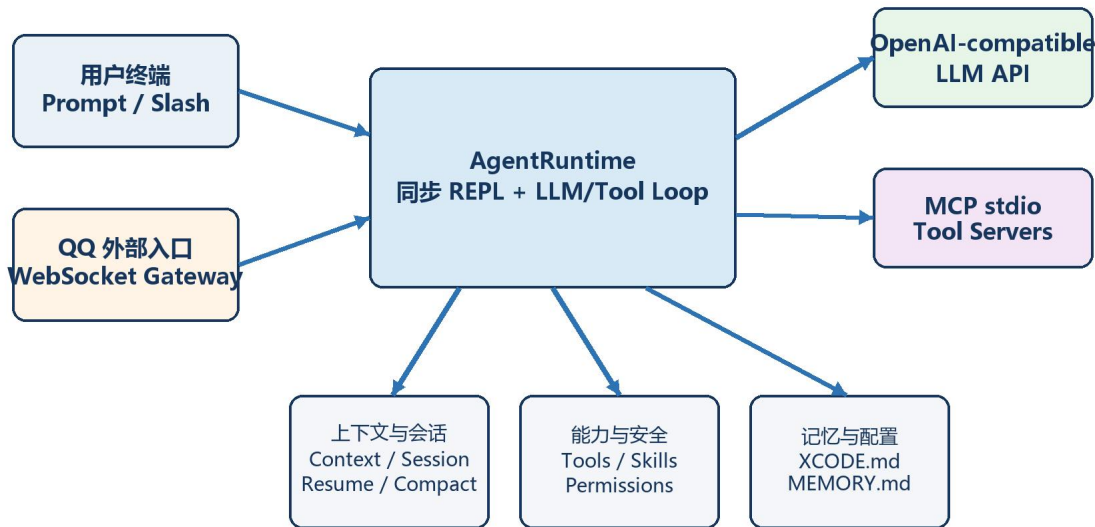


图 2 Xcode 核心架构：入口、运行时、领域子系统与外部服务

本地终端和 QQ 外部入口最终都进入统一的 LLM/tool loop，但二者不共享会话 history。AgentRuntime 在每轮中构造 system prompt、检查 context 阈值、调用 LLM；若模型返回 tool_calls，则 ToolCallExecutor 完成权限检查、执行与 transcript 写入，再携带结果继续下一轮，直到获得最终文本。

能力扩展: Skill、MCP 与 QQChat

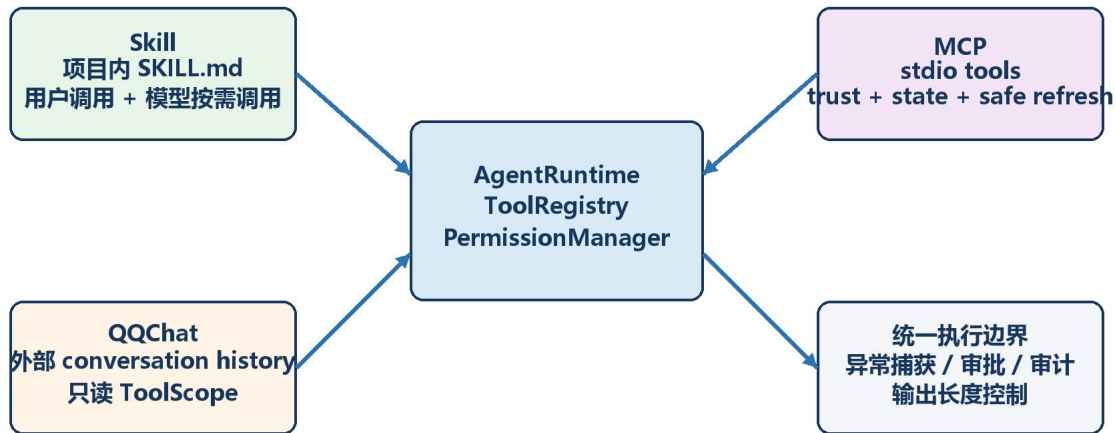


图 3 Skill、MCP 与 QQChat 的扩展关系

3.2 Memory 与会话数据流

文件驱动的分层 Memory 机制

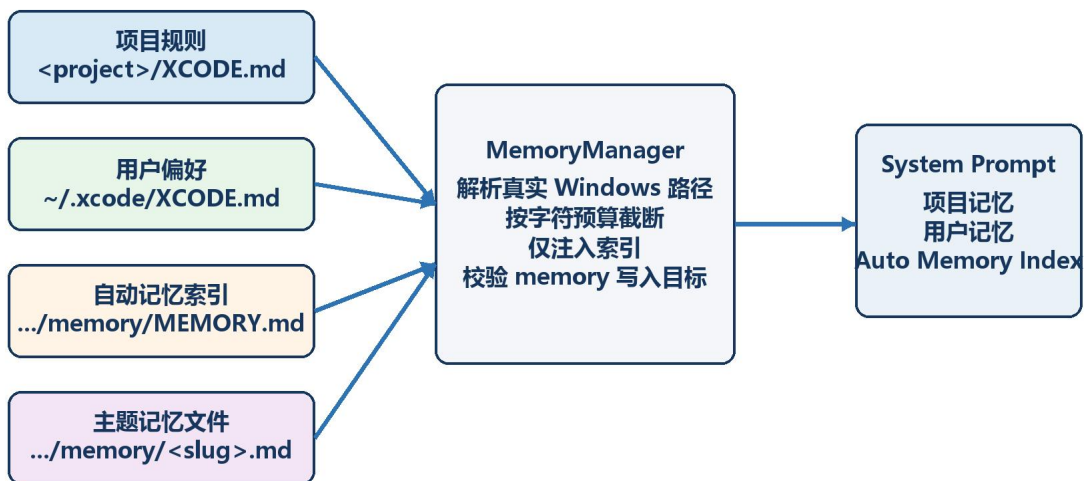
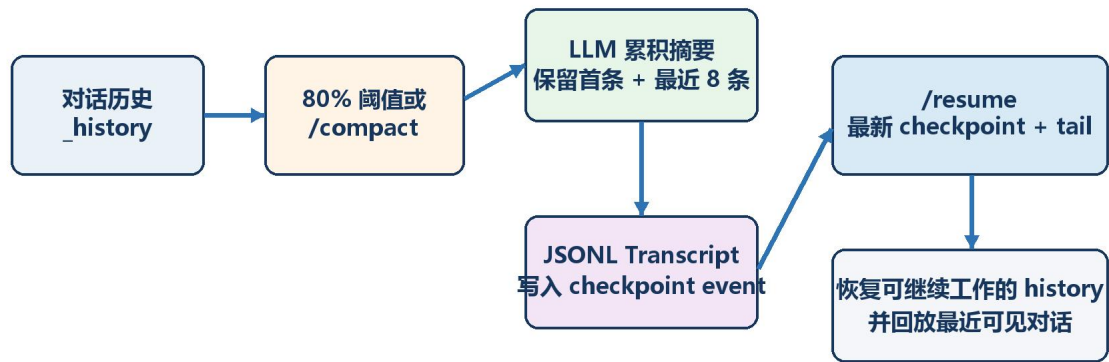


图 4 分层 Memory 与 Prompt 注入

Project XCODE.md 保存项目级规则，User XCODE.md 保存跨项目偏好，MEMORY.md 只作为主题记忆索引注入；详细主题文件按需 read_file，避免把全部长期记忆塞入上下文。MemoryManager 同时提供 resolved path，解决 Windows 盘符与模板路径误拼接问题。

Compaction Checkpoint 与 Session Resume



裁剪时保护 assistant tool_calls 与 tool result 配对，避免恢复非法消息序列。

图 5 /compact 与 /resume 的 checkpoint 数据流

SessionStore 保存 append-only JSONL 事实流，ContextManager 负责压缩策略，SessionResumeBuilder 负责从 transcript 重建模型 history。checkpoint 同时存在 system message 与独立 compaction_checkpoint event：前者服务模型，后者服务恢复和审计。

四、关键实现与代码展示

4.1 Agent 核心循环

Agent 核心循环采用“压缩检查 → LLM 调用 → 工具执行 → 历史追加 → 继续推理”的持续结构，不设置固定工具轮次上限。所有 LLM 与工具异常在边界处被转换为可读结果，避免单点失败终止 REPL。

```
while True:
    if self.context.should_compress(history):
        outcome = self.compactor.compact_history(history)
        history[:] = outcome.messages
        self.compactor.write_checkpoint(session_id, outcome)

    response = self.llm.complete(messages=history, tool_schemas=schemas)
    if not response.tool_calls:
        return response.content or "No response."

    tool_result = self.tool_executor.execute(response, tool_scope=tool_scope)
    history.append(tool_result.assistant_message)
    history.extend(tool_result.tool_messages)
```

代码来源: src/xcode_cli/core/agent.py:1069-1215 (节选整理)

4.2 Memory、/compact 与 /resume

Memory 采用文件模型而不是专用 CRUD 工具。这样既能复用 read/write/edit 工具，又能让记忆内容保持可读、可审计。MemoryManager 对三类注入内容分别截断，并对真正的 memory 目标提供路径范围判断；只有该范围内的写入可跳过常规 ask 审批，显式 deny 仍优先。

```
project = self._truncate(self.read_project_memory(), 2000)
user = self._truncate(self.read_user_memory(), 2000)
if self.is_auto_memory_enabled(cfg):
    index_content = self._truncate(self.read_memory_index(), 1200)
    context = "\n\n".join(blocks).strip()
    return self._truncate(context, 5000)
```

代码来源: src/xcode_cli/core/memory.py:62-82

压缩时保留第一条 user message 与最近 8 条消息，中间部分交给 LLM 生成累积摘要。摘要不仅保存需求与结论，也要求保留约束、文件变更、错误、待办和下一步。

```
compressed = []
if first_user:
    compressed.append(first_user)
checkpoint_message = {
    "role": "system",
    "content": f"Conversation summary checkpoint:\n{summary}",
```

```

}
compressed.append(checkpoint_message)
compressed.extend(tail)

```

代码来源: src/xcode_cli/core/context.py:124-137

恢复时扫描 transcript 中最后一个 compaction_checkpoint, 清空旧 tail, 只使用最新摘要和之后的 message events; 如果不存在 checkpoint, 则在 token budget 内从最近消息向前恢复。裁剪过程中会同步删除孤立的 tool result 或 tool_call, 保证消息序列合法。

4.3 Skill、MCP 与 QQChat

Skill 从项目 .xcode/skills/<name>/SKILL.md 加载。用户可以把 Skill 当作 slash prompt command, 模型也可以通过只读 skill 工具按需加载完整 prompt。系统只在 system prompt 中放轻量 listing, 从而避免每轮把所有 Skill 正文注入上下文; 模型调用后还会阻断同 turn 的再次 skill 调用, 防止递归。

```

return ToolDef(
    name="skill",
    parameters={"skill": {"type": "string"}, "args": {"type": "string"}},
    required=["skill"],
    execute=execute,
    is_read_only=True,
)
# execute() 返回 blocked_tools=["skill"]

```

代码来源: src/xcode_cli/core/tools/skill_tool.py:7-37

MCP 目前支持项目 .xcode/mcp.json 中的 stdio tools。连接前执行 trust fingerprint 校验, 连接后通过 tools/list 转换为 mcp_<server>_<tool>, 并在 AgentRuntime safe point 重建 ToolRegistry。单个 server 启动失败、schema 不兼容或调用超时均只影响对应状态, 不阻塞 Xcode 启动。

QQChat 通过 WebSocket Gateway 接收事件, 回调线程只完成 normalize、策略过滤、去重与入队; qqchat-worker 串行执行 ExternalTurnRunner。每个 QQ conversation key 维护独立 session/history, 默认只暴露 read_file、grep、glob、task_list, 并在 schema、allowlist、is_read_only 与 permissions 四层重复收口。

4.4 配置文件与 AI IDE 使用

配置/文档	作用	安全要点
~/.xcode/config.json	模型、token、渲染和 auto memory 等用户配置	敏感 API 信息仅保存在用户目录
.xcode/settings.json	项目级权限规则	显式 deny 优先
.xcode/mcp.json	MCP stdio server 与工具策略	secret 不进入 trust hash; 未 trust 不启动
.xcode/skills/*/SKILL.md	项目可复用工作流与 prompt	source hash 写入审计 metadata
docs/superpowers/specs / plans	Spec-first 设计与实施批次	实现结论必须跟在测试证据之后

五、测试与评估

5.1 自动化测试与 Benchmark

测试遵循 P0/P1/P2 风险分层，优先覆盖权限、工具循环、session resume、compaction checkpoint、memory 路径和外部入口安全。本报告编制时在 Windows、Python 3.10.9、提交 82b0a1d 上重新执行了相关测试。

验证项	命令/规模	结果	结论
重点机制回归	18 个相关测试文件	188 passed / 11.39 s	Memory、Resume/Compact、Skill、MCP、QQChat 主路径通过
全量测试基线	python -m pytest -q	508 passed / 24.85 s	未发现跨模块回归
Token 估算基准	1000 messages, 重复 200 次	平均 11.411 ms	本地估算开销远低于一次网络推理
Resume 基准	1000 events + checkpoint, 重复 100 次	平均 16.939 ms	恢复 401 条消息, 约 16245 tokens

代表性行为测试：

机制	测试重点	结果
Memory	项目/用户/索引注入顺序与截断；Windows 非法路径；memory-scoped 自动放行；显式 deny	通过
/compact	内容不足不写 checkpoint；累积摘要；Live 进度 finally 清理；自动/手动共用路径	通过
/resume	最新 checkpoint + tail；无 checkpoint fallback；工具消息配对；hidden skill prompt 不回放	通过

机制	测试重点	结果
Skill	frontmatter 加载、参数展开、模型调用校验、barrier、防递归、session audit	通过
MCP	trust、连接失败隔离、schema 转换、动态 refresh/reconnect、output limit、脱敏	通过
QQChat	事件归一化、策略过滤、去重、worker、不共享 REPL history、只读 ToolScope	通过

评估结论 当前实现已形成“可恢复、可压缩、可扩展、可审计”的闭环。性能瓶颈主要在模型与网络调用，而非本地 token 估算或 transcript 重建。剩余风险集中在第三方 MCP server 质量、QQ 网络稳定性以及更多终端环境的长期兼容。

5.2 Demo 截图占位

```
you ▶ /memory
Auto-memory: on
Project memory: D:\Xcode\XCODE.md (exists)
User memory: C:\Users\LONG\.xcode\XCODE.md (exists)
Memory dir: C:\Users\LONG\.xcode\projects\Xcode\memory
Memory files: 4 (index: 5 entries)
```

图 6 Memory 状态与文件内容

```
you ▶ /compact
Context compacted: 241 -> 12 messages, saved ~71932 tokens.
you ▶ /context
```

Item	Value
model	mimo-v2.5-pro
max tokens	128000
render mode	streaming_plus_final_render
syntax theme	monokai
messages	12
system prompt	~3818 tokens
user	1 msg / ~22 tokens
assistant	5 msg / ~1553 tokens
tool	4 msg / ~1813 tokens
history system	2 msg / ~1178 tokens
chat history	~4566 tokens
total	~8384 / 128000
free	~119616
compression threshold	~102400
auto-memory	on

图 7 /compact 进度与 checkpoint 结果



图 8 /resume 方向键菜单与最近对话回放

使用/test skill

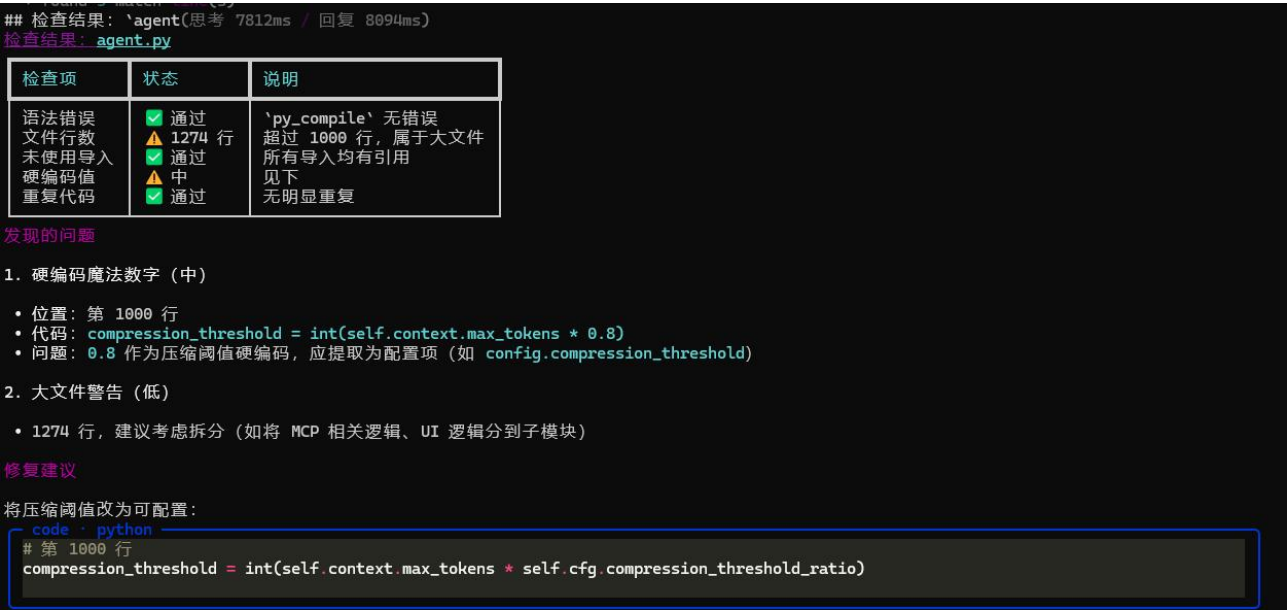


图 9 Skill 用户调用或模型主动调用

you > /mcp tools

MCP Tools

Server	Tool	Registered	State	Read-only	Output limit	Warnings
fake	echo	mcp__fake__echo	registered	False	20000 (config)	
fake	long_output	mcp__fake__long_output	registered	False	20000 (config)	
fake	fail	mcp__fake__fail	registered	False	20000 (config)	
sequential_thinking	sequentialthinking	mcp__sequential_thinking__sequentia...	registered	False	20000 (config)	

you > 你知道mcp__sequential_thinking__sequentia...是什么用法吗

你 你知道mcp__sequential_thinking__sequentia...是什么用法吗

assistant > 知道。这是我可用的一个 MCP 工具，用于**结构化链式思考**。

(思考 3467ms / 回复 7658ms)

知道。这是我可用的一个 MCP 工具，用于结构化链式思考。

用途

把复杂问题拆成多步思考链，每步可以：

- 建立在前一步基础上
- 质疑/修正之前的思路
- 分支探索不同方向
- 不确定时表达怀疑

典型场景

- 复杂问题需要多步推理
- 设计决策需要权衡
- 问题范围不确定，需要逐步澄清
- 需要生成假设并验证

参数核心

参数	说明
`thought`	当前思考步骤
`thoughtNumber`	当前步骤编号
`totalThoughts`	预估总步骤数（可动态调整）
`nextThoughtNeeded`	是否需要继续思考
`isRevision`	是否修正之前的思考
`branchFromThought`	从哪步分支

简单说

就是给自己一个“打草稿”的空间——在给出最终答案之前，先内部推理几轮。类似 Claude 的 extended thinking，但是显式的、可逐步调整的。

你想让我用它来分析某个问题吗？

图 10 MCP trust/status/tools 与工具执行

你愿意使用它来分析某个问题吗？

you > /QQ

Unknown command: /QQ. Try /help

you > /QQchat

QQchat

Key	Value
state	stopped
last_error	None
handled_messages	0
sent_replies	0
visible_tools	read_file, grep, glob, task_list
execution_allowlist	read_file, grep, glob, task_list
remote_approval	False

you > /QQchat start

QQchat

Key	Value
state	running
last_error	None
handled_messages	0
sent_replies	0
visible_tools	read_file, grep, glob, task_list
execution_allowlist	read_file, grep, glob, task_list
remote_approval	False

you >

你好，你是谁

你好小龙！我是**小米** (Xcode)，你正在开发的终端原生 AI 编码代理。

目前 v0.1.0, Phase 1-3 已完成，基于 Python + Typer + Rich + OpenAI API 构建。有什么需要我帮忙的吗？

帮我读一下当前项目的readme文档，简要说明一下

好的，以下是项目 README 的简要说明：

Xcode 项目概览

定位：终端原生 AI 编码代理（类似 Cloud Code），支持任何 OpenAI 兼容 API。

快速开始

```bash

pip install -e .

xcode

```

通过交互式 `/env` 命令或环境变量配置 API Key、Base URL 和模型名。

图 11 QQChat status 与真实消息 Demo

六、系统升级与扩展

Xcode 已通过明确的子系统边界为后续演进预留空间。下一阶段不应继续把功能塞回 AgentRuntime，而应围绕“统一 user turn、统一工具执行、统一持久化事实、入口级 ToolScope”四个稳定接口扩展。

阶段	升级内容	预期收益
近期	补充 CLI --resume / --continue; /context cost 估算；完善截图与录屏验收	提高可用性与课程展示完整度
会话能力	rollback、fork、命名 session、跨项目历史检索	支持并行探索与错误回退
Memory 2.0	主题生命周期、过期与冲突处理、可选向量检索、记忆来源引用	减少陈旧记忆与无关注入
Skill 生态	用户级/插件级 Skill、依赖检查、版本与签名、可控 fork context	形成可复用 Agent 工作流市场
MCP 生态	resources、prompts、HTTP/SSE transport、OAuth、tool search	接入更多企业系统并控制 schema 规模
外部入口	QQ 多会话持久化、审批回传、更多 IM 渠道适配	从本地助手演进为受控协作 Agent

AI 能力演进路径可概括为三个层次：第一层是“可靠执行”，确保工具异常、权限和会话状态正确；第二层是“长期协作”，通过 memory、resume 和 compact 保持跨时间连续性；第三层是“生态化智能”，通过 Skill 与 MCP 按需装载专业能力，并通过 QQChat 等外部入口服务真实协作场景。

架构原则 扩展能力必须先声明边界再接入：是否可见、是否可执行、是否只读、如何审批、如何持久化、失败是否可恢复。只有这些问题有明确答案，能力才应进入主运行时。

七、课程总结

7.1 个人收获

通过 Xcode 项目，我对 Agentic AI 的理解从“调用大模型生成代码”转变为“设计一个能够持续行动、保存状态并承担工程责任的软件系统”。真正困难的部分不是 prompt 本身，而是上下文、工具、权限、会话、异常和用户体验之间的协同。

Memory、/compact 与 /resume 的实现让我认识到，模型上下文不是数据库：短期 history、压缩摘要和长期 memory 必须分层；恢复也不是简单重放全部消息，而要维护 token 预算、工具消息配对和用户可见/模型可见内容的边界。

7.2 工程思维转变

项目采用 Spec-first + TDD-core + E2E-acceptance 流程后，我开始先定义行为与验收证据，再进入实现。对于权限、会话和外部入口等高风险功能，单元测试只能证明局部逻辑，仍需在 PowerShell/cmd.exe、真实 WebSocket 或 stdio 子进程环境中完成端到端验证。

我也更加重视“失败路径是一等公民”：工具、MCP server、QQ 网络或渲染组件都可能失败，但失败不应污染 history，更不能让整个 Agent 循环崩溃。这种面向边界和恢复性的设计思维同样适用于普通企业级应用。

7.3 对课程的建议

建议课程继续强化三类实践：一是要求学生提交可执行的 Product/Architecture/API Specs，而不只展示最终 Demo；二是增加权限、审计、成本和失败恢复等生产级指标；三是鼓励使用真实外部系统或 MCP 服务进行小规模集成，让学生体会 Agent 从实验原型走向企业应用时的工程约束。

总结 Xcode 的阶段性成果证明：Agentic AI 原生应用的核心竞争力，不只是模型能力，而是围绕模型建立的可靠运行时、上下文与记忆机制、安全工具系统以及可验证的工程流程。